Beyond Component Stacking: A Critical Review of Cross-Layer Evaluation for AI-Driven IoT Cybersecurity

Anomaly Detection, Adaptive Thresholding, Reinforcement-Learning Response, Post-Quantum Key Establishment, and Verifiable Audit Logs

Critical review manuscript draft | Evidence cutoff: 15 October 2026

Abstract

Internet of Things (IoT) cybersecurity increasingly combines machine-learning-based anomaly detection, automated response, cryptographic protection, and integrity-preserving audit records. Yet combining these components does not by itself demonstrate a stronger security system: the detector, decision threshold, response policy, communication channel, and evidence store can each fail under different assumptions, and the reported metrics are rarely directly comparable across papers. This critical review examines a focused corpus of research on LSTM and other deep-learning intrusion detectors, reinforcement-learning security policies, post-quantum cryptography (PQC) for constrained devices, and blockchain, hash-chain, and Merkle-tree logging. It uses Saeed's integrated LSTM/Q-learning/hash/PQC framework as a reference point and contrasts it with recent empirical work on CICIoT2023 and constrained-device deployment, recent mapping reviews of deep reinforcement learning, PQC surveys, and IoT provenance studies. The synthesis identifies five practical weaknesses that should guide implementation: insufficiently reproducible or non-public traffic setups; conflation of anomaly scores with operational alert decisions; weak control of benign drift and threshold contamination; RL evaluation that can reward containment while overlooking collateral blocking; and cryptographic or integrity claims that lack a precisely stated threat model. A cross-layer comparison schema and a reproducible experimental blueprint are proposed. The central recommendation is to prioritize a narrowly testable contribution—guarded adaptive thresholding under benign drift and malicious contamination—while treating response, PQC, and logging as separately measured integration layers. This creates a more defensible path to publication than adding components without isolating their contribution.

Index Terms—Internet of Things, intrusion detection, anomaly detection, adaptive thresholding, concept drift, reinforcement learning, post-quantum cryptography, ML-KEM, audit integrity, CICIoT2023.

1. Introduction

IoT deployments couple low-cost devices, gateways, edge services, and cloud platforms. They expose systems to availability attacks, botnets, reconnaissance, protocol abuse, and data manipulation, while imposing limits on memory, energy, compute, bandwidth, and maintenance. A practical defense is therefore a pipeline rather than a single classifier: traffic is transformed into features, a detector produces a score, a decision rule turns the score into an alert, a response policy chooses an action, cryptographic mechanisms protect communication, and logs preserve a verifiable record of what occurred.

Saeed [1] provides a direct reference architecture by combining LSTM reconstruction-based anomaly detection, a hidden-salt SHA-256 integrity check, Q-learning response, and lattice-based cryptography. The article is valuable because it treats detection, response, integrity, and future cryptographic resilience together. At the same time, the available description uses a simulated IoT stream of approximately 150,000 events with a 60/40 normal-to-attack mix and 30-second lookback windows, while the anomaly-detection results are presented largely as a demonstration of reconstruction behavior. This limits direct reproducibility and makes it difficult to infer how the design would compare under a public benchmark, strict temporal or device-grouped splits, and a common metric set. These are review findings about reporting and comparability, not evidence that the proposed components cannot work.

The component literature has advanced considerably. Banaamah and Ahmad [3] compare CNN, LSTM, and GRU detectors; Yaras and Dener [4] evaluate a 1D CNN-LSTM hybrid on CICIoT2023 and TON_IoT; and Mazinani et al. [6] compare LSTM, GRU, RNN, CNN, TCN, and MLP models on CICIoT2023, including analysis on constrained hardware. In response policy, Uprety and Rawat [2] survey reinforcement learning for IoT security, while two newer reviews focus on DRL-based IoT intrusion detection and adversarial RL [7], [8]. PQC also has dedicated reviews, including a 2026 systematic review covering IoT PQC challenges [9] and an earlier performance-and-optimization survey [10]. For evidence integrity,

blockchain provenance, signature chains, and deterministic Merkle trees have all been studied [13]-[15]. A review must therefore avoid simply listing algorithms or asserting that the combination is novel without comparing prior integrated designs. An adjacent 2025 review considers IoT anomaly detection in relation to quantum machine learning [19]; that detection-oriented direction should be distinguished from post-quantum cryptography, which concerns cryptographic protection against quantum-capable adversaries.

This review takes an implementation-oriented view. It asks not only which technique is used, but also what input it receives, what security decision it makes, which threat model it assumes, what is measured, and whether an independent researcher could reproduce the result.

1.1 Research questions

- RQ1: Which anomaly-detection model and sequence-construction choices are defensible for public IoT traffic datasets, and which metrics support fair model selection?
- RQ2: How do threshold adaptation and traffic drift affect the operational false-alarm rate after a detector has produced anomaly scores?
- RQ3: What must be measured to show that an RL response policy improves containment without unacceptable false blocking or availability loss?
- RQ4: How should PQC key establishment and tamper-evident logging be benchmarked separately from per-event detection latency?
- RQ5: What minimum common protocol would make an integrated IoT cybersecurity framework reproducible and meaningfully comparable with earlier work?

1.2 Contributions

- A cross-layer taxonomy separating data construction, anomaly scoring, thresholding, response, secure transport, and evidence integrity.
- A source-grounded comparison of representative empirical studies, surveys, a public dataset paper, and the current NIST ML-KEM standard.
- A comparability checklist that prevents unsupported rankings across different datasets, splits, class definitions, and threat models.
- A prioritized implementation agenda identifying guarded adaptive thresholding under drift and contamination as the most focused experimental contribution to pursue first.

2. Review scope and method

This is a structured critical review and implementation-oriented synthesis, not a completed systematic literature review. The starting corpus comprised 14 full-text PDFs supplied for this task: studies on deep-learning IDS, RL and adversarial RL, PQC, blockchain-backed logs, provenance, and deterministic Merkle-tree verification. The core integrated framework by Saeed [1] was additionally checked against its public IEEE bibliographic record and paper text. Targeted source checks were also made for the original CICIoT2023 dataset publication, the NIST FIPS 203 standard, and PRISMA 2020 guidance [16]-[18]. The evidence cutoff is 15 October 2026.

The included source types serve different purposes. Primary empirical papers are used to understand model choices, datasets, metrics, and deployment setups. Review papers are used to map algorithm families, repeated limitations, and research directions. The NIST standard is used to state what ML-KEM actually standardizes; PRISMA is referenced as a reporting guideline for a future systematic expansion. Review-level study counts are quoted only as reported by their original authors and are not combined into a new PRISMA flow count.

Because the search process was seeded by the supplied files and targeted searches rather than a fully logged multi-database screening exercise, the present paper does not claim exhaustive coverage or report a PRISMA diagram. Before submission as an SLR, the authors should search databases such as IEEE Xplore, ACM Digital Library, Scopus or Web of Science, ScienceDirect, SpringerLink, and Google Scholar with date-stamped Boolean queries; deduplicate records; record inclusion/exclusion decisions; and publish the extraction sheet and screening log. PRISMA 2020 supports transparent reporting of why a review was done, how sources were selected, and what was found [18].

2.1 Inclusion logic and extraction fields

A source was relevant when it provided a technical method, empirical evaluation, or synthesis directly related to at least one layer in the pipeline. For each source, the review extracted: publication type and year; task and threat model; dataset or environment; algorithm; training and testing assumptions when stated; metrics; implementation/deployment evidence; and the main limitation relevant to reproduction or cross-paper comparison. Missing values are treated as not reported, not inferred.

2.2 Comparability rule

Reported metrics are not pooled or ranked across papers unless dataset, label definition, data split, task (binary versus multiclass), and metric formula are demonstrably compatible. A high accuracy from a balanced binary task is not directly comparable to macro-F1 on an imbalanced 34-class task. Similarly, an offline model F1 does not establish real-time mitigation benefit, and a cryptographic microbenchmark does not establish that an end-to-end security system is faster or safer.

3. Cross-layer view of IoT cybersecurity

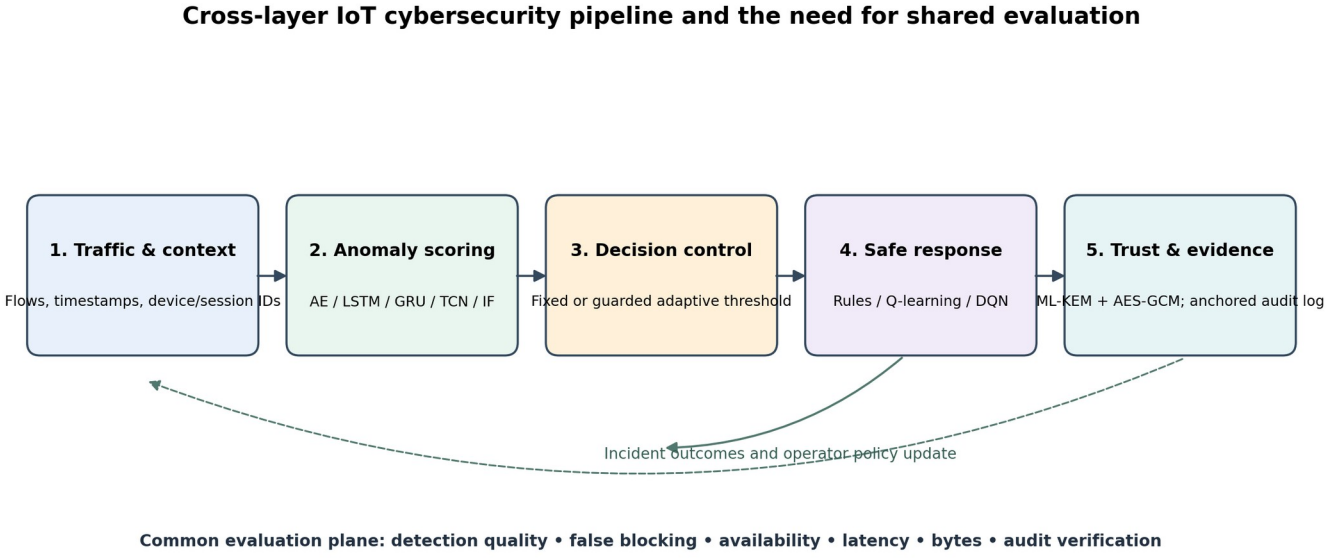


Fig. 1. Cross-layer pipeline and shared evaluation plane. Each module has a distinct security property and must be measured on its own as well as in the integrated system.

The most consequential distinction is between an anomaly score and an alert decision. A detector may output a reconstruction error e_t, probability, distance, or novelty score. The threshold τ_t converts this continuous score into a binary event: $a_t = 1[e_t > \tau_t]$. Changes to τ_t can alter precision, recall, and the false-positive rate without changing the model weights. The response policy then consumes the alert and operational context; its quality depends on both detection behavior and action costs. This suggests that detector architecture, threshold update rule, and response policy must be evaluated as separate variables before their interactions are evaluated.

The secure-transport and evidence modules solve different problems. PQC is relevant to key establishment and cryptographic resilience; a KEM such as ML-KEM establishes a shared secret that can then be used by symmetric authenticated encryption. It is not itself the bulk-data cipher. An audit structure such as a hash chain or Merkle tree supports detection of modifications only under a specified trust and anchoring model. These functions should not be collapsed into one generic ‘security score.’

4. Evidence synthesis by component

4.1 Anomaly detection: model choice is inseparable from data construction

Deep-learning IDS studies commonly consider CNN, LSTM, GRU, hybrid CNN-LSTM, and autoencoder approaches [3]-[6]. The choice should follow the data and task rather than the name of the architecture. LSTM and GRU are appropriate

when the input is a real ordered sequence with meaningful temporal dependencies. A tabular flow row that has been shuffled, or a window that mixes different devices or sessions, does not become a valid temporal sequence merely because it is reshaped into a tensor. Such sequence construction can make recurrent models learn accidental dataset ordering rather than network behavior.

Mazinani et al. [6] provide particularly relevant deployment evidence: their study uses CICIoT2023 and compares LSTM, GRU, RNN, CNN, TCN, and MLP models, evaluates binary and multiclass settings, and measures model complexity and execution on a constrained STM32H7S78-DK board. Their reported trade-off analysis favors MLP and TCN for practical deployment; post-training INT8 quantization reduces model size by more than 90% with negligible performance degradation in the reported setting. This is an important warning against assuming that an LSTM is automatically the best choice just because it appears in an integrated framework.

Yaras and Dener [4] report a 1D CNN-LSTM hybrid using both CICIoT2023 and TON_IoT, with correlation-based feature reduction and comparisons to ten ML/DL methods. They report very high accuracy, including 99.995% in binary CICIoT2023 classification and 99.96% in multiclass classification. These should be recorded as author-reported results, not directly ranked against other papers without checking task construction, class distributions, splits, and metrics. The MQTT-oriented study by Vijayan and Sundar [5] uses MQTTSet and compares combinations of autoencoder, deep belief network, LSTM, and heuristic feature selection, illustrating that protocol-specific IoT traffic may call for different datasets and assumptions.

The key evaluation lesson is to distinguish at least three tasks: (i) supervised intrusion classification, where labeled attacks are available during training; (ii) unsupervised or semi-supervised anomaly detection, where training is restricted to benign examples or weak labels; and (iii) sequence prediction or reconstruction, where the validity of temporal order is central. A supervised XGBoost reference can be useful as a performance ceiling or task reference, but it is not a like-for-like competitor to a normal-only autoencoder. The paper must identify the difference explicitly.

4.2 Threshold adaptation: the often-missing decision layer

Fixed percentile thresholds are easy to reproduce and form essential baselines, but they assume the calibration distribution remains relevant. Under benign distribution shift, a fixed threshold may cause false alarms to rise; under an adaptive threshold, unguarded updates may instead absorb sustained attack-related scores and raise the threshold until attacks are missed. These are different failure modes and require different experiments. In Saeed [1], the text itself identifies fixed-threshold limitations and suggests dynamic thresholding or hybrid methods as future work; therefore, threshold adaptation is a direct extension opportunity, but it is not novel merely by being added.

Useful comparator families include: (a) fixed validation-calibrated percentiles; (b) rolling quantile thresholds; (c) mean/variance tracking such as EWMA; (d) robust statistics such as median and median absolute deviation; and (e) guarded adaptation that freezes or limits threshold updates when the stream appears suspicious. A defensible new method should specify exactly when updates are permitted, how quickly the threshold can move, how attacks are prevented from contaminating the baseline, and how the detector resumes adaptation after an incident. ‘Adaptive thresholding’ without a defined update rule and threat model is too vague to replicate.

A practical contribution is a bounded, contamination-gated threshold controller. Let e_t be the detector score, τ_t the current threshold, and c_t a gate indicating whether the recent stream is eligible for adaptation. The baseline rule is $a_t = 1[e_t > \tau_t]$. Update the calibration buffer only when $c_t = 1$, the score is safely below the alarm boundary, the local alert rate is not elevated, and no incident is active. Winsorize extreme accepted scores and clip the maximum per-step movement $|\tau_{t+1} - \tau_t|$ to a configured bound. Freeze adaptation during candidate attacks and require a clean cooldown before resuming. Use separate alarm and release thresholds (hysteresis) to avoid rapid alert flapping. Each safeguard introduces parameters that must be pre-registered and ablated; it must not be represented as a proven improvement until it is measured.

4.3 Reinforcement learning: containment must be constrained by availability

RL represents an adaptive defense problem as states, actions, transitions, and rewards. Uprety and Rawat [2] map RL across IoT and cyber-physical security, including the exploration and convergence difficulties of large state spaces. The 2026 systematic mapping review by Sawad et al. [7] reports 267 articles identified and 26 retained after screening for DRL-based IoT intrusion detection; DQN appeared in eight of the 26 included studies (30.8%). That result illustrates that DQN is a common design, not that DQN is uniformly superior. Abu Al-Haija and Al Tamimi [8] broaden the discussion to adversarial RL and adversarial training, including the robustness of policies under hostile conditions.

For an implementation paper, Q-learning and DQN should be compared against a transparent non-learning baseline under the same held-out test episodes. State variables must be available at inference time and must not contain the ground-truth test label. Actions should be operationally safe and auditable, for example alert, rate-limit, quarantine, temporarily isolate, or block a specifically defined flow under an explicit policy. A vague or simulated ‘counterattack’ action is not a deployable defense action and creates safety and evaluation ambiguity.

The reward should penalize collateral damage as well as missed attacks. Report containment rate with a denominator and time horizon, false-block rate on benign flows, service availability, time-to-mitigation, response latency, and cumulative reward. A policy that blocks every device can achieve apparent containment while destroying availability. RL claims also require disclosure of the environment, reward values, state/action definitions, training episodes, random seeds, and policy selection rule.

4.4 PQC: benchmark a standard KEM, not a vague ‘quantum encryption’ block

PQC literature for IoT emphasizes that cryptographic agility and quantum resistance must be balanced against device compute, memory, bandwidth, energy, and integration constraints. Liu et al. [10] survey performance and optimization for resource-constrained devices and note that optimization proposals have lacked standardization. Asif [11] and Rubia et al. [12] survey PQC families and IoT implementation constraints; Adere et al. [9] provide a recent systematic review of IoT PQC threats, countermeasures, key-management concerns, scalability, and integration issues.

The current practical reference is NIST FIPS 203, which specifies ML-KEM-512, ML-KEM-768, and ML-KEM-1024 as module-lattice-based key-encapsulation parameter sets [17]. A KEM establishes a shared secret; a symmetric AEAD such as AES-GCM encrypts and authenticates application data using a derived key. Thus, an implementation should separately measure key generation, encapsulation, decapsulation, derived-key setup, per-record AEAD, serialized key/ciphertext bytes, memory, and (if available) energy. Report the library, version, CPU/device, compiler/build flags, parameter set, warm-up, number of repetitions, and p50/p95 latency. The first-handshake cost must not be reported as though it were incurred for every packet if keys are reused for a session.

Bit-flip tests showing rejection are useful functional tests but do not constitute a cryptographic security proof. A PQC implementation claim should state the standard and parameter set, use reviewed libraries, verify failure handling and key lifecycle, and avoid claiming universal quantum-proofness. NIST’s FIPS 203 page should be checked for current errata before implementation, since the standard page records a note that an issue has been identified for future correction [17].

4.5 Logs and evidence integrity: tamper evidence depends on an external trust anchor

Hsu et al. [13] propose an IoT log storage protocol combining blockchain, a signature chain, and attribute-based access control, with formal/security analysis and performance considerations. Sigwart et al. [14] present a generic blockchain-based provenance framework built with Ethereum smart contracts and discuss extensibility alongside scalability and privacy constraints. Liu et al. [15] describe a more recent deterministic-Merkle-tree framework for heterogeneous IoT logs, including canonicalization, batch roots, external anchoring, audit paths, inclusion/consistency verification, and tampering localization.

These designs solve related but non-identical goals. A hash chain provides a simple way to detect changes in sequence if a trusted checkpoint survives. A Merkle tree supports efficient inclusion proofs and batch-level verification. HMAC authenticates data to parties that hold a shared secret; digital signatures support public-key verification and attribution. Blockchain may distribute trust but adds storage, consensus, privacy, latency, or operational costs. None should be described as absolutely immutable without defining the adversary’s control. If a privileged attacker can rewrite the log and recompute all hashes, a local chain alone is insufficient; externally anchored, signed, or independently stored checkpoints change that threat boundary.

For an incident-management system, log at least the event identifier, device or flow pseudonym, timestamp, detector version, score, threshold version, action, action outcome, cryptographic key identifier (never the secret key), and integrity-chain or Merkle-batch identifier. Avoid unnecessarily recording raw payloads or personal data. Evaluate tampering by mutation type: edit a field, delete a record, reorder records, insert a fabricated record, truncate the tail, and rewrite a whole local chain before/after an external anchor. Report detection coverage and localization granularity, not only ‘100% tamper detection’ for a narrow set of bit flips.

5. Comparative map of representative sources

Table 1 summarizes the uploaded and directly relevant sources. The comparison is qualitative by design because the papers use different datasets, tasks, threat models, and validation protocols.

Source	Method / focus	Evaluation context	Value and key limitation
[1] Saeed, 2025	Integrated LSTM anomaly detection, Q-learning response, salted SHA-256 check, lattice/LWE-based cryptography	Simulated IoT stream described as approx. 150k events, 60/40 normal/attack, 30-second windows	Useful integration reference; needs public benchmark reproduction, common metrics, and more explicit per-component protocols
[3] Banaamah & Ahmad, 2022	CNN, LSTM, GRU IDS models	Empirical IoT IDS benchmark; task-specific settings	Supports architecture comparisons, but does not evaluate RL, PQC or evidence logging
[4] Yaras & Dener, 2024	1D CNN-LSTM hybrid and ten comparison methods	CICIoT2023 and TON_IoT; binary and multiclass	Public-data comparison; high reported accuracy is not directly rankable without matching split/task definitions
[5] Vijayan & Sundar, 2023	Autoencoder/DBN and LSTM/DBN with heuristic feature selection	MQTTSet	Useful protocol-specific detector example; different traffic/task limits direct comparison
[6] Mazinani et al., 2026	LSTM, GRU, RNN, CNN, TCN and MLP; tuning, INT8 quantization	CICIoT2023; binary/8-class/34-class; STM32H7S78-DK analysis	Strong deployment-aware evidence; authors find MLP/TCN favorable on their trade-off score, but no PQC/RL/logging integration
[2] Uprety & Rawat, 2021	Survey of RL/DRL for IoT and CPS security	Literature review	Useful RL taxonomy; not a common empirical benchmark
[7] Sawad et al., 2026	Systematic mapping of DRL-based IoT IDS	267 identified; 26 retained, per authors	DQN appears in 8/26 included studies; frequency is not proof of superiority
[8] Abu Al-Haija & Al Tamimi, 2026	Adversarial RL and NIDS survey	Literature review	Adds robustness and adversarial-training lens; does not rank policies on a shared testbed
[10] Liu et al., 2024	PQC performance and optimization survey	Resource-constrained-device literature	Useful hardware/latency/optimization taxonomy; update implementation to current standards
[9] Adere et al., 2026	PRISMA-based IoT PQC review	5,151 initial records; 161 retained, per authors	Recent taxonomy for threats, key management, scalability and integration; PQC-only scope
[11] Asif, 2021	Lattice-based PQC survey for IoT	Literature review	Background on lattice methods and constraints; current standards should take precedence in implementation
[12] Rubia et al., 2024	Broad PQC family survey	Literature review	Useful taxonomy but less implementation-specific than the current standard
[13] Hsu et al., 2020	Blockchain, signature chain and attribute-based access control for IoT logs	Protocol and security analysis	Relevant to evidence, authentication and privacy; trust and resource assumptions differ from a local hash chain
[14] Sigwart et al., 2024 issue	Generic Ethereum smart-contract data provenance	Proof-of-concept, performance/requirements discussion	Useful provenance architecture; authors discuss scalability and privacy trade-offs
[15] Liu et al., 2026	Canonical logs, deterministic Merkle roots, external anchors and audit paths	Recent framework/design paper	Useful design for inclusion/consistency verification and tamper localization; benchmark costs against alternatives

5.1 What cannot be concluded from the comparison

- The highest accuracy reported in a supervised benchmark does not establish that a normal-only anomaly detector is inferior; training information differs.
- A recurrent network trained on correctly ordered sessions is not comparable to a recurrent network fed shuffled rows or windows that cross devices/sessions.
- A reported end-to-end F1 does not establish that a thresholding strategy or RL component caused the gain unless controlled ablations isolate it.
- Crypto setup latency and per-event AEAD latency are different quantities; the number of handshakes and key reuse policy affect system-level cost.
- Hash-chain or Merkle tamper tests are only as strong as their adversary model, canonicalization rules, and independently protected checkpoint/anchor.

6. The research gaps that matter for implementation

6.1 Gap A: lack of a common cross-layer evaluation protocol

The reviewed sources evaluate different layers in isolation or use incompatible test setups. The integrated base paper [1] demonstrates a combined design, while component studies and recent reviews examine model architectures, DRL policies, PQC, or evidence logs independently [6]-[15]. Within this focused corpus, a common protocol that reports anomaly detection, threshold robustness, response safety, PQC overhead, and audit verification on a shared event trace was not identified. This is a bounded-corpus observation, not proof that no such study exists anywhere.

Research implication: a useful contribution is not a single composite score. It is a reproducible benchmark protocol in which every component receives the same defined events and the output of each layer can be traced. Report detection performance, response utility, communication overhead, and evidence correctness separately, then describe end-to-end behavior.

6.2 Gap B: score-to-decision behavior under drift and contamination

Thresholding is an attractive core contribution because it is measurable, relatively cheap to implement, and tightly linked to operational false alarms. A valid experiment must separately manipulate benign traffic drift and malicious score contamination. Scaling feature values is not the same as increasing packet rate or changing a device's benign behavior distribution; these scenarios should be named and generated separately. A threshold controller can fail by adapting too slowly, adapting to an attack, or adapting in a way that suppresses recall. The experiment should show all of these possibilities rather than designing only a favorable drift scenario.

6.3 Gap C: safe response policy evaluation

A response model needs an explicitly defined operational objective. Containment alone is inadequate. The evaluation must expose the cost of false blocking, loss of service, action oscillation, and response latency. Static policy baselines must be tuned under the same constraints, not intentionally weakened. RL policies should train on separate environment episodes and evaluate on held-out attack scenarios; test labels may be used for scoring but never as inputs to the deployed state or policy.

6.4 Gap D: cryptography is often described at a higher level than it is measured

'Post-quantum encryption' may refer to a KEM, a signature, a hybrid key exchange, or a bespoke scheme; those operations are not interchangeable. A defensible implementation should state the selected standard and distinguish the key-establishment phase from symmetric data encryption. The same applies to integrity: salted hashing, keyed MACs, signatures, hash chains, Merkle inclusion proofs, and blockchain provenance make different guarantees.

6.5 Gap E: audit integrity requires a realistic privileged-attacker model

Local tampering tests are a start but should not be the entire security evaluation. The threat model should state whether the attacker can modify a device, gateway, database, log writer, hash store, key store, time source, and external anchor. If all integrity state is held by one compromised host, the attacker may be able to recompute it. Remote checkpoints, protected signing keys, immutable storage, and independent audit nodes reduce—but do not eliminate—specific risks.

6.6 Gap F: system overhead should be budgeted by phase

Detection inference, response computation, KEM handshake, per-record encryption, audit append, and integrity verification should each be timed separately. An integrated number is meaningful only when the event path and measurement boundaries are explicit. If the KEM handshake is once per session, its cost should be amortized over the session in an additional reported view, not silently added to or removed from per-event latency.

7. Recommended implementation blueprint

The following blueprint is intentionally designed to be implementable by a small research team. It prioritizes one principal hypothesis while still permitting a credible integrated demonstration. The review does not claim that the proposed method has already improved results; the method and hypotheses below must be tested.

7.1 Recommended principal contribution

Recommended paper direction: ‘Contamination-Resistant Adaptive Thresholding for Unsupervised IoT Intrusion Detection under Benign Drift, with Risk-Constrained Incident Response.’ The new research claim should be about a well-specified and validated threshold controller, not about adding LSTM, RL, PQC, and logs together. Use a simple detector and an end-to-end prototype to show system relevance, but keep cryptography and logging as engineering components unless their implementation itself is a research contribution.

7.2 Data protocol

- Use CICIoT2023 [16] as the main public dataset. Record the exact release, files, feature schema, filtering and label mapping. If an external validation dataset is added, treat it as a separate domain-generalization experiment rather than pooling its records into the same random split.
- Split by time, device, capture, or attack scenario before constructing temporal windows. Never allow overlapping windows from the same underlying record region to cross train, validation, and test partitions. If a fully chronological/grouped split is infeasible because of dataset structure, justify and disclose the alternative.
- Fit imputers, scalers, encoders, feature selectors and threshold-calibration parameters on training/validation partitions only. Report duplicate or near-duplicate overlap across partitions; deduplication sensitivity is a check, not proof that all leakage is absent.
- For an unsupervised autoencoder, train on benign training data only. Use validation data to choose architecture and threshold. Keep test labels out of model, threshold-update, and RL state decisions; labels are reserved for final scoring.
- Preserve true temporal order for LSTM/GRU/TCN experiments. Group by device/session/flow as appropriate; provide the sequence length, stride, padding policy, and handling of incomplete windows. Compare tabular models on tabular features without artificially imposed order.

7.3 Model and threshold baselines

Use a small, interpretable model matrix to keep the implementation manageable:

Role	Recommended methods	Fairness condition
Unsupervised detector	Dense autoencoder; Isolation Forest	Same benign-only training partition and same held-out evaluation stream
Sequence detector	LSTM-AE or GRU-AE; optionally TCN	Valid chronological/session windows; report sequence construction and memory/latency
Reference only	XGBoost or another supervised classifier	Clearly label as supervised; not a like-for-like unsupervised model
Threshold baselines	Fixed q95, fixed q99, rolling quantile, raw EWMA	Same detector scores and same test stream; calibration on validation only
Proposed threshold	Guarded adaptive quantile/statistics with bounded updates, incident freeze, hysteresis	Pre-register parameters; ablate each guard; test both benign drift and attack contamination

7.4 Proposed guarded threshold update

Let e_t denote the anomaly score and τ_t the alert threshold. The detector emits an alert if $e_t > \tau_t$. A separate eligibility gate g_t decides whether the score stream can update the baseline. A practical candidate design is:

- Initialize τ from a fixed validation-calibrated percentile or robust score statistic.
- Define an alert band and a release band (hysteresis) so that small score fluctuations do not repeatedly toggle the response.
- Update the rolling baseline only when no incident is active, the alert rate is not elevated, and recent scores are safely below the alarm boundary; do not update on points near the threshold.
- Winsorize score contributions and bound threshold movement per step; log each update reason, old threshold, new threshold and detector version.
- Freeze updates during sustained high-score bursts, repeated alerts, or suspected contamination. Resume only after a prespecified clean cooldown or controlled operator release.
- Evaluate failure cases deliberately: benign rate/feature drift, bursty benign traffic, gradual score drift, sustained attack contamination, alternating attack/benign periods, and attacks whose scores lie just below the alarm boundary.

This controller is a candidate design, not a guaranteed defense. The acceptance gate can itself be evaded by low-and-slow attacks; the guard mechanisms should therefore be assessed both alone and in combination. Compare the full method with fixed thresholds, raw EWMA, and a simple rolling quantile to show whether the added complexity earns its cost.

7.5 RL response design

Use static rules as the minimum baseline; add tabular Q-learning before DQN if the state space is small enough. A minimal runtime state can include anomaly score band, recent score trend, repeated-alert count, device/service criticality, and the confidence of supporting signals. The state must be built only from observations available at inference time. Actions should be bounded by an explicit policy: alert, rate-limit, quarantine, isolate, or block under authorization. Train using training scenarios and choose the policy on validation episodes; hold out entire attack scenarios or device groups for final evaluation.

Reward should reflect multiple costs, for example $r_t = B_c \cdot \text{contained}_t - C_b \cdot \text{false_block}_t - C_a \cdot \text{availability_loss}_t - C_l \cdot \text{latency}_t$, with weights stated and sensitivity-tested. The weights cannot be chosen after inspecting test results. Report containment and false-blocking separately so that trade-offs remain visible.

7.6 PQC and evidence logging design

Use a standards-based ML-KEM parameter set (for example ML-KEM-768 as a primary configuration, with ML-KEM-512 and ML-KEM-1024 as sensitivity configurations) following FIPS 203 [17]. Use the KEM result to derive a session key and protect event payloads with AES-256-GCM or a selected standard AEAD. Report key-setup timings separately from steady-state encryption/decryption.

For logging, compare (i) an unkeyed hash chain with an independently protected checkpoint, (ii) HMAC-authenticated records or chain links, and (iii) batched Merkle roots with externally anchored roots. Avoid raw blockchain unless the trust distribution justifies the added cost. Test edit/delete/reorder/insert/truncate/rewrite attacks and state which adversary capabilities each method resists. Include record size and proof/checkpoint costs.

7.7 Hypotheses and outcomes

Hypothesis	Independent variable	Primary outcomes	What would count as evidence
H1: guarded adaptation is more robust under benign drift	Threshold method × drift type/severity	FPR, recall, F1, detection delay, threshold movement	Lower FPR than static threshold under benign drift while preserving attack recall; no claim if one improves only by sacrificing the other
H2: the simplest detector may be competitive at lower cost	Detector architecture under same data split	Macro-F1, PR-AUC, FPR, inference latency, RAM/model size	Quality-cost trade-off measured on the same data and, ideally, same hardware
H3: RL can improve containment under a false-block constraint	Static rule vs Q-learning vs DQN	Containment, false-block rate, availability, response latency	Improvement persists on held-out scenarios at a prespecified false-block/availability limit
H4: PQC cost is concentrated by phase	ML-KEM parameter set and session policy	Keygen/encap/decap p50/p95, bytes, RAM, per-event AEAD latency	Initial setup and steady-state data-plane cost are reported separately

Hypothesis	Independent variable	Primary outcomes	What would count as evidence
H5: batch proof structures can lower verification/record overhead	Hash chain vs HMAC chain vs Merkle batch	append/verify throughput, bytes/record, proof size, attack coverage	Measured trade-off under identical records and explicit anchor/key threat model

7.8 Metrics and statistical reporting

For detection, report precision, recall, F1, macro-F1 for multiclass settings, PR-AUC, ROC-AUC, FPR, and false alarms per fixed number of benign events. Report per-attack-family recall if labels support it. For drift experiments, report detection delay, threshold adaptation lag, maximum threshold excursion, FPR increase, and recovery time after the drift ends. For defense, define the numerator and denominator of containment and false-block metrics, then report service availability and response latency. For PQC and logs, report p50/p95 latency, throughput, serialized bytes, memory, and power/energy if measured.

Repeat stochastic training and environment simulations across multiple predeclared seeds. Report mean and standard deviation or confidence intervals; use paired comparisons when the same seeds/scenarios are used. Specify hardware and software versions. Keep raw per-seed rows in the repository, generate figures from those rows, and ensure that manuscript tables are generated from the same source data. Do not write ‘proves’ based only on a small non-significant difference, and do not combine results from incompatible evaluation protocols into a leaderboard.

8. Publication strategy: how to be more than the base paper

A broad framework paper is attractive because it addresses many layers, but it is difficult to establish novelty when every layer uses an established method and the evaluation does not isolate their interactions. A stronger follow-on implementation paper should state one main claim and design every experiment around falsifying it.

8.1 Most feasible path

The highest-priority implementation direction from this review is a controlled study of contamination-resistant, drift-aware thresholding for unsupervised IoT anomaly detection, with optional risk-constrained response as a secondary demonstration. This is easier to implement and validate than inventing new detector, RL, PQC, and audit algorithms simultaneously. It is also directly motivated by the base paper's fixed-threshold limitation and the review's analysis of model/threshold separation [1], [6], [7].

Proposed implementation-paper title: ‘Contamination-Resistant Adaptive Thresholding for Unsupervised IoT Intrusion Detection under Benign Drift and Adversarial Score Contamination.’ The title should be retained only if the implementation actually evaluates both benign drift and attack contamination. A safe-response module can then be used to show the operational relevance, but it should not obscure the primary threshold hypothesis.

8.2 Minimum experiment package for a credible paper

- One public primary dataset (CICIoT2023) and a fully documented split/preprocessing pipeline; add TON_IoT or MQTTSet only if time permits and task harmonization is valid.
- One main detector plus one lightweight baseline and one supervised reference; correct temporal modeling only where data are genuinely sequential.
- At least four threshold strategies: fixed q95, fixed q99, raw EWMA or rolling quantile, and the guarded proposal; include threshold-parameter sensitivity.
- Separate clean/benign-drift/attack-contamination scenarios, varying drift magnitude, duration, and attack score profile.
- Five or more seeds, per-seed result files, paired comparison, runtime cost, and ablations of update gate, clipping, incident freeze, and hysteresis.
- A reproducibility package with scripts, exact configuration, environment file, hashes of input files, machine-readable metrics, and figure/table generators.

8.3 Claims to avoid

- ‘Our framework outperforms the base paper’ unless the baseline is reproduced or adapted under a common public dataset, threat model, and metric protocol.

- ‘Homomorphic hashing’ for ordinary salted SHA-256. A salted hash is not automatically a homomorphic hash; name the operation according to its actual cryptographic property.
- ‘100% tamper-proof’ or ‘quantum-proof’. State the tested attacks, algorithm/standard, parameter set, and limits of the threat model.
- ‘Real-time’ based only on average desktop latency. Define the hardware, workload, queueing, p95 latency and throughput target.
- ‘Zero-day detection’ based only on random splits where instances from the same attack campaign or capture may occur in both training and testing. Reserve attack families or time periods for a stronger novelty test.

9. Limitations of this review

This review is bounded by the full-text documents supplied and targeted source verification, and it is not an exhaustive systematic search. The component studies are heterogeneous in datasets, class labels, attack types, hardware, split design, and reported metrics. Several studies report high accuracy, but the present review does not independently reproduce them and does not pool their scores. The base paper's simulator and reported outputs were analyzed as published, not independently rerun. The recommended adaptive-threshold controller is a research design proposal that requires empirical validation. The 2026 Merkle-tree source is treated as a recent design contribution; any benchmark claims should be verified against the paper's full reported experimental evidence before being used as quantitative evidence.

For an actual systematic review submission, expand the search, use an explicit screening protocol, complete the PRISMA 2020 checklist and flow diagram where applicable, and release the extracted evidence table. For a conventional critical review, the current scope and limitations should remain explicit so that the manuscript does not overstate its coverage.

10. Conclusion

The evidence supports a layered interpretation of AI-driven IoT cybersecurity. Deep learning provides anomaly scores; threshold control determines when scores become alarms; RL maps alarms and context to defensive action; PQC protects key establishment; and audit structures support evidence verification. Each is necessary for a different security objective, and none can stand in for another. The central methodological issue is not whether all components can be combined, but whether their individual and joint effects can be measured reproducibly under a common threat model.

The most practical route toward a stronger implementation paper is to focus on the score-to-decision boundary: test a bounded, contamination-gated adaptive threshold under realistic benign drift and malicious contamination, using a public IoT dataset, correct temporal splits, transparent baselines, and metrics that expose false alarms and delayed detection. Then evaluate response safety and measure PQC and audit overhead by phase. This narrower claim will be easier to falsify, easier for others to reproduce, and easier to distinguish from a framework that primarily assembles established components.

References

[1] M. M. Saeed, “An AI-Driven Cybersecurity Framework for IoT: Integrating LSTM-Based Anomaly Detection, Reinforcement Learning, and Post-Quantum Encryption,” IEEE Access, vol. 13, pp. 104027–104036, 2025, doi: 10.1109/ACCESS.2025.3576506.

[2] A. Uprety and D. B. Rawat, “Reinforcement Learning for IoT Security: A Comprehensive Survey,” IEEE Internet of Things Journal, early access, doi: 10.1109/JIOT.2020.3040957, 2021.

[3] A. M. Banaamah and I. Ahmad, “Intrusion Detection in IoT Using Deep Learning,” Sensors, vol. 22, no. 21, Art. no. 8417, 2022, doi: 10.3390/s22218417.

[4] S. Yaras and M. Dener, “IoT-Based Intrusion Detection System Using New Hybrid Deep Learning Algorithm,” Electronics, vol. 13, no. 6, Art. no. 1053, 2024, doi: 10.3390/electronics13061053.

[5] P. M. Vijayan and S. Sundar, “An automated system of intrusion detection by IoT-aided MQTT using improved heuristic-aided autoencoder and LSTM-based Deep Belief Network,” PLOS ONE, vol. 18, no. 10, e0291872, 2023, doi: 10.1371/journal.pone.0291872.

[6] A. Mazinani, D. Antonucci, L. Davoli, and G. Ferrari, “Performance Assessment of DL for Network Intrusion Detection on a Constrained IoT Device,” Future Internet, vol. 18, no. 1, Art. no. 34, 2026, doi: 10.3390/fi18010034.

[7] M. O. A. Sawad, S. J. Abdulkadir, H. S. Alhussian, and M. M. Eltahir, “Deep Reinforcement Learning-Based Intrusion Detection in IoT Networks: A Systematic Mapping and Literature Review,” Computer Modeling in Engineering & Sciences, vol. 148, no. 2, 2026, doi: 10.32604/cmes.2026.085472.

[8] Q. Abu Al-Haija and S. Al Tamimi, “A State-of-the-Art Survey of Adversarial Reinforcement Learning for IoT Intrusion Detection,” Computers, Materials & Continua, vol. 87, no. 1, 2026, doi: 10.32604/cmc.2025.073540.

[9] K. Adere et al., “Systematic review on securing IoT systems with post quantum cryptography emerging threats, countermeasures, and future research needs,” Discover Internet of Things, vol. 6, Art. no. 14, 2026, doi: 10.1007/s43926-025-00275-6.

[10] T. Liu, G. Ramachandran, and R. Jurdak, “Post-Quantum Cryptography for Internet of Things: A Survey on Performance and Optimization,” arXiv:2401.17538, 2024. [Online]. Available: https://arxiv.org/abs/2401.17538.

[11] R. Asif, “Post-Quantum Cryptosystems for Internet-of-Things: A Survey on Lattice-Based Algorithms,” IoT, vol. 2, no. 1, pp. 71–91, 2021, doi: 10.3390/iot2010005.

[12] J. Rubia, R. Babitha Lincy, E. E. Nithila, S. Shibi, and R. Rosi, “A Survey about Post Quantum Cryptography Methods,” EAI Endorsed Transactions on Internet of Things, vol. 10, 2024, doi: 10.4108/eetiot.5099.

[13] C.-L. Hsu, W.-X. Chen, and T.-V. Le, “An Autonomous Log Storage Management Protocol with Blockchain Mechanism and Access Control for the Internet of Things,” Sensors, vol. 20, no. 22, Art. no. 6471, 2020, doi: 10.3390/s20226471.

[14] M. Sigwart, M. Borkowski, M. Peise, S. Schulte, and S. Tai, “A secure and extensible blockchain-based data provenance framework for the Internet of Things,” Personal and Ubiquitous Computing, vol. 28, pp. 309–323, 2024 (first published online 2020), doi: 10.1007/s00779-020-01417-z.

[15] D. Liu, F. Li, and S. Fang, “A framework for verifying the integrity of IoT logs and tracing tampering based on deterministic Merkle trees,” Advanced Electromagnetics, vol. 15, no. 3, 2026, doi: 10.7716/aem.v15i3.4245.

[16] E. C. Pinto Neto et al., “CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment,” Sensors, vol. 23, no. 13, Art. no. 5941, 2023, doi: 10.3390/s23135941.

[17] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS PUB 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203.

[18] M. J. Page et al., “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,” BMJ, vol. 372, n71, 2021, doi: 10.1136/bmj.n71.

[19] A. J. Aparcana-Tasayco, X. Deng, and J. H. Park, “A systematic review of anomaly detection in IoT security: towards quantum machine learning approach,” EPJ Quantum Technology, vol. 12, Art. no. 112, 2025, doi: 10.1140/epjqt/s40507-025-00414-6.

Appendix A. Compact checklist before submitting the implementation paper

Check	Minimum evidence
Research claim	One main hypothesis; baselines and negative outcomes are stated before testing.
Data provenance	Exact public source/version, labels, filters, features, transformations and script versions.
Split discipline	No overlapping window/capture leakage; validation used for tuning; test labels do not affect updates.
Temporal validity	Real order/session grouping; sequence length, stride and boundary handling documented.
Threshold fairness	Fixed q95/q99, rolling/ EWMA baseline and proposed guarded controller use the same scores/stream.
Drift taxonomy	Benign feature/distribution drift, benign traffic-rate burst and attack score contamination are separate cases.
RL safety	State/action/reward; train/validation/test episodes; false-block and availability constraints; safe actions only.
PQC validity	FIPS 203 parameter set; library/version; KEM versus AEAD clearly distinguished; handshake and per-event cost separated.
Log integrity	Hash/HMAC/Merkle properties defined; anchors/keys protected; mutation and truncation attacks tested.
Reporting	Per-seed raw CSV; metrics generated from scripts; p50/p95 and uncertainty; machine/hardware disclosed.
Reproducibility	README, environment lockfile, configs, exact run commands, raw result artifacts and figure/table generators.
Claim discipline	No superiority claim against Saeed [1] without common-protocol reproduction or clearly labeled adaptation.